

# ZooKeeper: coordinación wait-free para sistemas distribuidos

López Arteaga, César Omar

Universidad Nacional de Ingeniería - Facultad de Ciencias  
Ciencias de la Computación

Junio 2026

# Agenda

Problema

Contribución

Modelo de datos

API y garantías

Watches

Implementación

Recetas

Evaluación

Análisis crítico

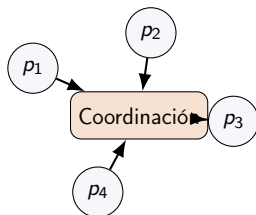
Conclusión

# Problema de coordinación distribuida

## Pregunta central

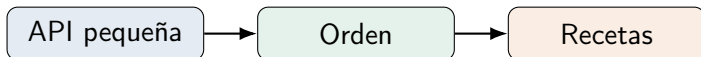
¿Cómo coordinar miles de procesos distribuidos sin que locks bloqueantes o clientes lentos degraden todo el sistema?

- Configuración dinámica
- Membresía de grupo
- Elección de líder
- Detección de fallos
- Sincronización



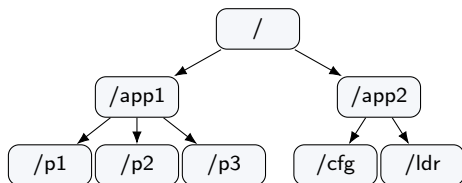
## Contribución del paper

- ▶ **Núcleo de coordinación:** API simple con objetos wait-free.
- ▶ **Recetas en cliente:** locks, líder, membresía y barreras se construyen fuera del servidor.
- ▶ **Alto rendimiento:** lecturas locales; escrituras ordenadas por Zab.
- ▶ **Garantías clave:** FIFO por cliente y escrituras linealizables.



# Znodes: árbol jerárquico de metadatos

- ▶ Cada nodo se llama **znode**.
- ▶ Almacena datos y metadatos.
- ▶ Tipos: regular, ephemeral y sequential.
- ▶ Diseñado para coordinación, no para almacenar archivos grandes.



# API esencial

| Operación                               | Uso principal                              |
|-----------------------------------------|--------------------------------------------|
| <code>create(path,data,flags)</code>    | Crear znode regular, efímero o secuencial. |
| <code>delete(path,version)</code>       | Eliminar con control de versión.           |
| <code>getData(path,watch)</code>        | Leer datos y registrar watch.              |
| <code>setData(path,data,version)</code> | Escritura condicional.                     |
| <code>getChildren(path,watch)</code>    | Listar hijos y observar membresía.         |
| <code>sync(path)</code>                 | Sincronizar antes de una lectura fuerte.   |

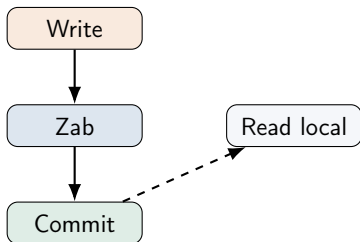
# Garantías de consistencia

## Escrituras

Linealizables: todas las actualizaciones se ordenan globalmente.

## Cliente

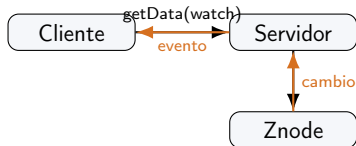
FIFO: las operaciones de un mismo cliente respetan el orden de envío.



Lectura rápida; usar sync si se necesita precedencia fuerte.

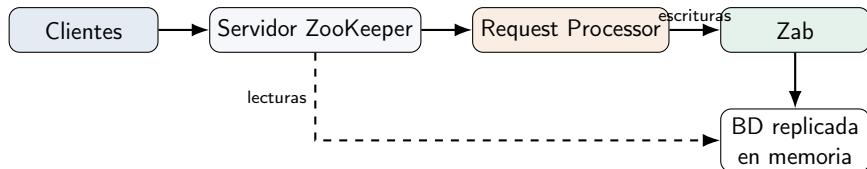
## Watches: notificación sin polling

- ▶ Se registran en operaciones de lectura.
- ▶ Son disparadores de una sola vez.
- ▶ Notifican que algo cambió, no entregan el nuevo valor.
- ▶ Reducen polling y favorecen caché en cliente.





# Arquitectura del servicio



- ▶ El líder ordena las escrituras mediante Zab.
- ▶ Cada réplica responde lecturas desde memoria.
- ▶ Log + snapshots permiten recuperación.

## Recetas de coordinación

| Receta            | Construcción con ZooKeeper                          |
|-------------------|-----------------------------------------------------|
| Configuración     | Znode de configuración + watches.                   |
| Membresía         | Znodes efímeros bajo un directorio de grupo.        |
| Elección de líder | Znodes secuenciales; gana el menor número.          |
| Lock sin manada   | Cada cliente observa solo su predecesor.            |
| Barrera doble     | Entrada y salida controladas por hijos de un znode. |

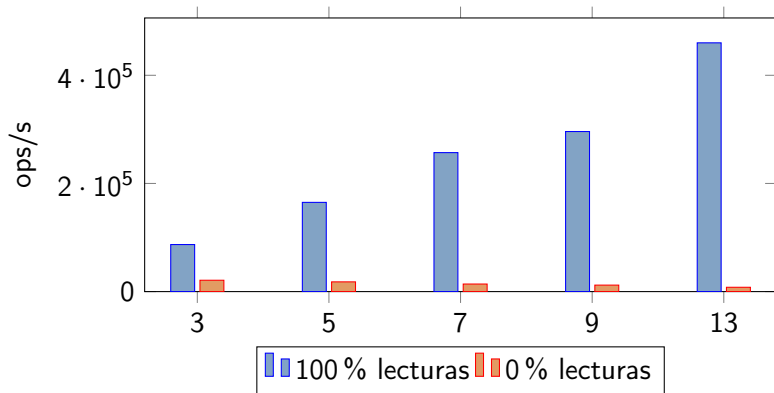
## Lock sin efecto manada



### Clave

Al liberar lock-001, solo despierta el cliente que observaba lock-001. No se despiertan todos los clientes.

## Resultados de throughput



# Fortalezas y límites

## Fortalezas

- ▶ API mínima.
- ▶ Buen rendimiento en lecturas.
- ▶ Sesiones y znodes efímeros.
- ▶ Recetas flexibles.

## Límites

- ▶ Lecturas pueden ser obsoletas.
- ▶ Requiere quorum.
- ▶ No almacena datos grandes.
- ▶ Recetas correctas dependen del cliente.

# Conclusión

## Mensaje principal

ZooKeeper demuestra que un servicio pequeño, replicado y con garantías de orden cuidadosamente elegidas puede resolver gran parte de la coordinación distribuida real.

- ▶ Las escrituras se ordenan globalmente.
- ▶ Las lecturas se escalan localmente.
- ▶ Los clientes componen primitivas más fuertes.
- ▶ El enfoque wait-free mejora rendimiento y tolerancia a clientes defectuosos.

Referencia: Hunt, Konar, Junqueira y Reed, *ZooKeeper: Wait-free coordination for Internet-scale systems*.